



Agentic AI: From Traditional Software Systems to Autonomous AI Agents Systems

The AI Agentic Revolution

- The LLM Generative AI movement has been a great breakthrough that is creating many openings and many great innovative developments
- One of these great innovations is the Agentic AI paradigm which is drastically changing the way we build and apply software systems
- The Autonomous AI Agent is the central feature and player in this paradigm
- Instead of building instruction-based components and objects, we build dynamic intelligent autonomous agents that perceive their environment, reason, plan and take decisions without intervention from human beings

A Turning Point in the History of Computing

- We are witnessing a pivotal turning point in the history of computing.
- The transition from traditional software systems to autonomous agents represents a fundamental paradigm shift that transforms how digital systems operate and interact with their environments.
- While conventional programs operate within predetermined pathways defined by explicit instructions, agent-based systems exhibit goal-directed behavior, maintain persistent state, and adapt their strategies based on environmental feedback.

New Frameworks and Different Architectures

- This transformation challenges established software engineering principles and introduces new frameworks for conceptualizing intelligence in computational systems.
- The distinction between traditional software and agent-based approaches is not merely semantic but architectural.
- While conventional systems process discrete inputs to generate predictable outputs, agents operate continuously within dynamic environments, forming internal representations, making decisions under uncertainty, and learning from experience.

Reconceptualization and New Intelligent Behaviors

For practitioners trained in deterministic programming models, this shift requires not only new technical skills but a reconceptualization of how intelligent systems function and evolve. Key traits that distinguish intelligent agents from traditional software include:

- **Autonomy:** The ability to operate without continuous human guidance.
- **Persistence:** Maintaining state and memory across interactions.
- **Reactivity:** Responding to changes in the environment in real time.
- **Proactiveness:** Initiating actions based on internal goals, not just external triggers.
- **Adaptability:** Learning from experience and modifying behavior accordingly.
- **Goal-orientation:** Pursuing objectives through planning and reasoning under uncertainty

What are AI Agents?

- *“An AI agent is a computational system that perceives its environment, processes internal state, and takes actions to achieve defined goals.*
- *These systems exhibit autonomy, adaptability, and reactivity—key attributes that differentiate them from traditional software programs.*
- *An agent operates not merely by reacting to inputs, but by maintaining context, managing goals, and adjusting strategies based on feedback.*
- *This dynamic behavior draws from the paradigm of **situated AI**, where intelligence emerges from continuous interaction with the environment. “*

According to an old Franklin and Graesser definition:

“An autonomous agent is a system situated within and a part of an environment that senses environment and acts on it, over time, in pursuit of its own agenda”

The Implication of the Definition

This definition laid the groundwork for architectures that incorporate sensing, planning, acting, and learning.

In enterprise applications, agents are increasingly deployed as digital workers—handling customer onboarding, processing invoices, managing workflows—each with persistent state, memory, and feedback mechanisms.

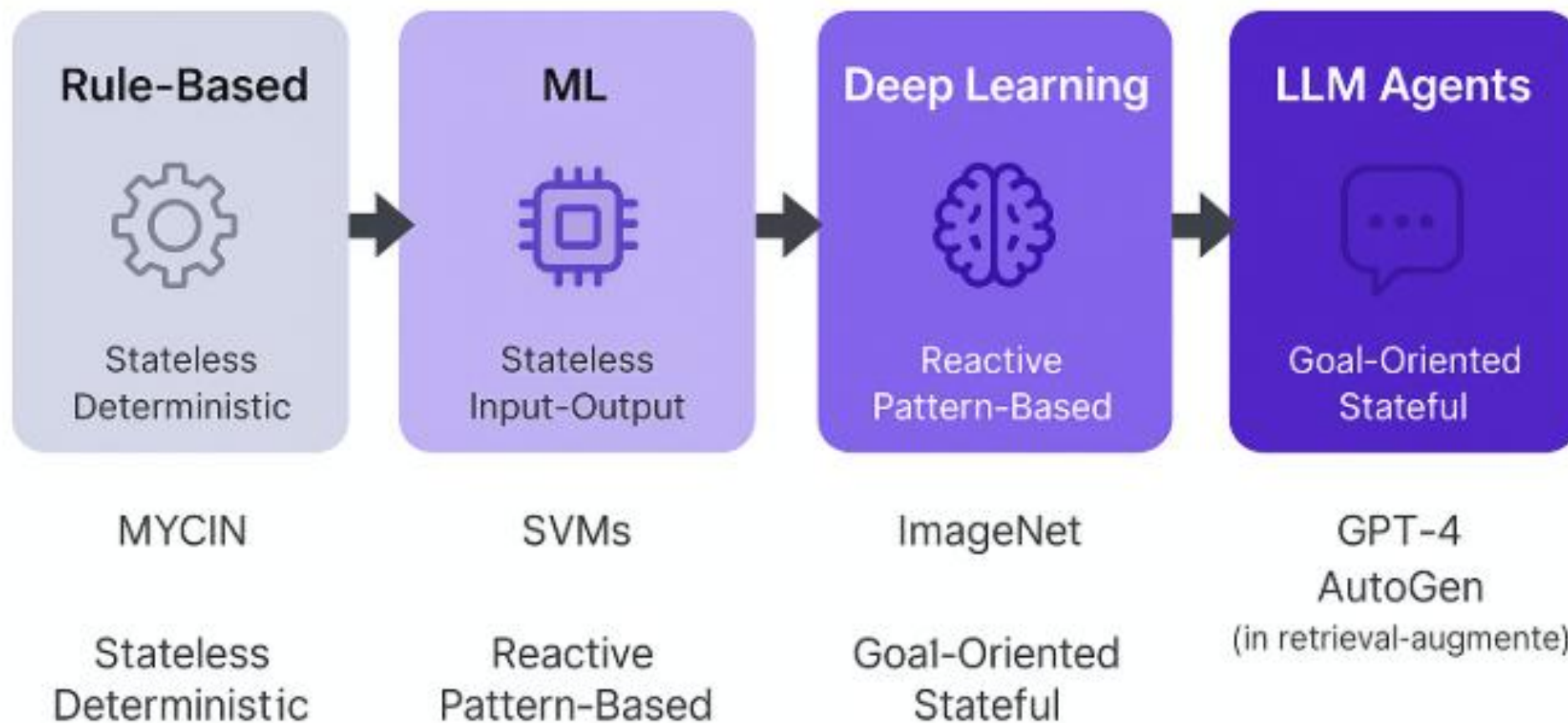
A Little Bit of History

The history of AI agent development can be segmented into distinct technological eras:

- **1970s–1980s:** Rule-based expert systems, such as MYCIN (a Stanford-developed system for diagnosing blood infections and recommending antibiotics), used logic-based inference engines to solve narrowly defined problems. Despite deterministic precision, these systems were brittle and inflexible.
- **1990s:** Classical machine learning methods like decision trees and SVMs introduced pattern recognition capabilities. While more adaptive than rule systems, they remained task-specific and stateless.
- **2010s:** Deep learning revolutionized data perception. Speech recognition, image analysis, and translation reached human-level performance. However, these models were largely reactive, designed for input-output prediction rather than autonomous behavior.
- **2020s and beyond:** The advent of **large language models (LLMs)** — AI systems trained on vast text datasets to understand and generate human language — and **transformers** — neural network architectures that excel at processing sequential data — introduced emergent reasoning, natural language generation, and few-shot learning. Yet early LLMs were limited by context size, lack of memory, and tool integration.

Evolutionary Trajectory

- This evolutionary trajectory, illustrated in *Figure below (next slide)* highlights fundamental architectural and philosophical distinctions between conventional AI applications and truly autonomous agent systems—
- differences that extend well beyond technical implementation to how these systems operate, learn, and interact with their environments.
- These architectural shifts are not just academic—they translate into measurable business outcomes such as reduced support costs, increased first-contact resolution rates, faster onboarding, and greater scalability across customer touchpoints.



Architecture of Agents

- The architectural design of intelligent agents marks a fundamental shift from procedural logic to cognition-driven computation.
- Unlike traditional software systems that execute static instructions in response to defined inputs, agents operate continuously within dynamic environments, making real-time decisions, maintaining persistent memory, and adapting their strategies over time.
- At its core, an agent's architecture must integrate key cognitive functions—*perception, reasoning, planning, action, and learning*—into a modular, stateful framework that supports both reactivity and deliberation.

Separating Concerns to Realize Cohesion

- The architectural cohesion is what transforms agents from reactive bots into intelligent systems capable of navigating open-ended, real-world complexity
- In practice, cohesion is realized by designing systems that separate concerns:
 - **perception** modules interface with sensors or APIs;
 - **planning engines** decompose objectives;
 - **memory subsystems** manage historical and semantic context;
 - **execution layers** interface with tools, services, or other agents.
- Frameworks like **LangGraph** and **CrewAI** implement these principles by providing composable runtime environments where agents can maintain state across sessions, orchestrate workflows using graphs, and operate autonomously.

The Cognitive Loop

- The cognitive architecture of intelligent agents defines how perception transforms into purposeful action through structured, repeatable processes.
- At the heart of this design lies the **cognitive loop**—a continuous cycle of perception, reasoning, planning, action, and learning—which enables agents to operate autonomously in dynamic environments.
- As illustrated in the *Figure Below*, this loop forms the backbone of intelligent agent behavior, providing the scaffolding through which decisions are made, actions are executed, and knowledge is accumulated over time



Phases of the Cognitive Systems

To understand how this architecture functions in practice, let's explore each phase of the cognitive loop in detail—beginning with *perception*, the critical first step that shapes everything that follows

Perception

Perception initiates the loop by capturing data from the environment—whether through user input, APIs, sensors, or external systems—and converting it into structured formats suitable for processing. This raw input forms the basis for subsequent cognitive steps and determines the scope of the agent's situational awareness.

Example: Perception in a customer service agent

```
def perceive_input(user_message, context):  
    return {  
        "message": user_message,  
        "timestamp": datetime.now(),  
        "user_id": context.get("user_id"),  
        "session_state": context.get("session"),  
        "sentiment": analyze_sentiment(user_message)  
    }
```

Reasoning

Reasoning follows by contextualizing this perceived information, applying pattern recognition, inference engines, or statistical models to extract meaning and relevance. This stage transforms signals into insights, allowing the agent to understand not just what is happening, but why it matters.

```
# Example: Reasoning about customer intent
def reason_about_intent(perception_data):
    intent = classify_intent(perception_data["message"])
    priority = determine_priority(
        intent,
        perception_data["sentiment"],
        user_history=get_user_history(perception_data["user_id"])
    )
    return {"intent": intent, "priority": priority, "context": percep}
```

Planning

Planning orchestrates these insights into a coherent sequence of actions. Whether using deterministic rule chains or probabilistic models, the agent decomposes objectives into tasks, evaluates options, and prioritizes steps in accordance with predefined goals and environmental conditions.

Example: Planning response strategy

```
def create_action_plan(reasoning_result):  
    if reasoning_result["intent"] == "billing_issue":  
        return [  
            "fetch_account_details",  
            "analyze_billing_history",  
            "generate_explanation",  
            "offer_resolution"  
        ]  
    elif reasoning_result["priority"] == "urgent":  
        return ["escalate_to_human", "log_urgent_case"]
```

Action

Action then executes the selected steps, interfacing with external tools, APIs, databases, or systems to operationalize the agent's decisions. This phase is often implemented using function-calling frameworks or tool orchestration layers such as those found in LangChain or LangGraph.

Example: Action execution

```
def execute_action(action_plan, context):  
    results = []  
    for action in action_plan:  
        if action == "fetch_account_details":  
            result = billing_api.get_account(context["user_id"])  
        elif action == "generate_explanation":  
            result = llm.generate_response(context, results)  
        results.append(result)  
    return results
```

Learning

- **Learning** closes the loop by analyzing outcomes, measuring the success of actions, and updating internal models or memory stores. This feedback mechanism allows the agent to refine its behavior over time, improving performance based on both successes and failures.

-

Example: Learning from interaction

```
def learn_from_outcome(interaction_data, user_feedback):  
    success_score = calculate_success(user_feedback)  
    update_user_preferences(interaction_data["user_id"], success_score)  
    if success_score < 0.7:  
        flag_for_model_improvement(interaction_data)
```